

Week 4 Assignment

Travel Expense Tracker API

1. Project Overview

During a trip, multiple friends spend money on food, travel, accommodation, shopping, tickets, and other activities. Usually, different people pay for different expenses, which makes it difficult to track how much each person has actually spent and how much they owe to others.

In this project, you will build a Travel Expense Tracker API that allows a group of friends to record, manage, and split their expenses during a trip.

The application will be developed as a backend REST API using Python, FastAPI, and MongoDB.

The main goal is to get practical experience with:

- FastAPI
- REST APIs
- Pydantic models
- MongoDB
- CRUD operations
- Query parameters
- Filtering and sorting
- Basic calculations
- MongoDB aggregation
- Input validation
- Error handling

2. Objective

Develop a backend system where users can:

- Create and manage trips
- Add friends to a trip
- Record expenses
- Specify who paid for an expense
- Specify who shared the expense
- Calculate individual shares
- View trip expenses
- Filter and search expenses
- Generate a trip-wise expense summary
- Understand who has paid more and who needs to pay others

3. Example Scenario

Suppose four friends go on a trip:

Anil, Ravi, Suresh, and Kiran

During the trip:

- Anil pays ₹1,200 for dinner

- Ravi pays ₹800 for travel
- Suresh pays ₹400 for tickets
- Kiran pays ₹600 for food

For the dinner:

Total Amount: ₹1,200

Paid By: Anil

Shared By: Anil, Ravi, Suresh, Kiran

Each person's share is:

$$₹1,200 / 4 = ₹300$$

Since Anil paid ₹1,200 but his own share is only ₹300, the remaining ₹900 should be recovered from the other three members.

The application should help calculate these amounts.

4. Core Features

A. Trip Management

The system should allow users to create a trip.

A trip may contain:

- Trip name
- Destination
- Start date
- End date
- List of friends

Example:

Trip Name : Goa Trip

Destination: Goa

Members : Anil, Ravi, Suresh, Kiran

Students can decide how they want to structure the trip document in MongoDB.

B. Add Expense

Users should be able to add an expense to a particular trip.

An expense should contain information such as:

- Expense title
- Amount
- Category
- Paid by
- People sharing the expense
- Date
- Optional description

Example:

Title : Dinner

Amount : 1200

Category : Food

Paid By : Anil

Shared By : Anil, Ravi, Suresh, Kiran

Date : 2026-08-08

Description : Dinner at restaurant

Suggested categories:

- Food
- Travel
- Accommodation
- Shopping
- Entertainment
- Tickets
- Other

Students may add their own categories.

5. Expense CRUD Operations

The application must support basic CRUD operations.

Create

Add a new expense.

POST /trips/{trip_id}/expenses

Read

View all expenses for a trip.

GET /trips/{trip_id}/expenses

View a particular expense.

GET /expenses/{expense_id}

Update

Update an existing expense.

PUT /expenses/{expense_id}

Delete

Delete an expense.

DELETE /expenses/{expense_id}

6. Expense Splitting

The application should calculate the individual share of an expense.

For example:

Expense Amount = ₹1,000

Number of People = 4

Individual Share = ₹250

If:

Paid By = Ravi

Shared By = Ravi, Anil, Suresh, Kiran

Then:

Ravi → Paid ₹1,000 | Share ₹250

Anil → Paid ₹0 | Share ₹250

Suresh → Paid ₹0 | Share ₹250

Kiran → Paid ₹0 | Share ₹250

The application should be able to determine the amount each person owes or is entitled to receive.

7. Trip Summary

Create an API that provides a summary of the trip.

Example:

GET /trips/{trip_id}/summary

The summary should include information such as:

Total Trip Expenses

Total Amount Paid by Each Person

Individual Share

Amount Owed

Amount to Receive

Example:

Total Trip Expense: ₹8,000

Anil:

Paid : ₹4,000

Share : ₹2,000

Balance : +₹2,000

Ravi:

Paid : ₹1,000

Share : ₹2,000

Balance : -₹1,000

Suresh:

Paid : ₹2,000

Share : ₹2,000

Balance : ₹0

Kiran:

Paid : ₹1,000

Share : ₹2,000

Balance : -₹1,000

Here:

- Positive balance → Person should receive money
- Negative balance → Person needs to pay money
- Zero balance → Person is settled

The exact settlement logic and implementation approach are left to the students.

8. Search, Filter and Sort

The API should support basic querying.

Students should implement filtering based on one or more of the following:

Category

GET /trips/{trip_id}/expenses?category=Food

Person

GET /trips/{trip_id}/expenses?paid_by=Anil

Date

Allow filtering expenses based on date.

Amount

Allow filtering based on minimum or maximum amount.

Example:

GET /trips/{trip_id}/expenses?min_amount=500

Sorting

Allow expenses to be sorted by:

- Amount
- Date

Example:

GET /trips/{trip_id}/expenses?sort_by=amount

Students should make use of MongoDB queries for these operations.

9. MongoDB Requirements

MongoDB must be used as the database.

Students should design their own suitable database structure.

The database should contain information related to:

- Trips
- Members/Friends
- Expenses

Students should make appropriate use of MongoDB features such as:

- Documents
- Arrays
- Nested objects
- Query operators
- Sorting
- Aggregation

The exact schema is not fixed. Students should justify their design choices.

10. Suggested Expense Structure

Students may use a structure similar to:

```
{
  "title": "Dinner",
  "amount": 1200,
  "category": "Food",
  "paid_by": "Anil",
  "shared_by": [
    "Anil",
    "Ravi",
    "Suresh",
    "Kiran"
  ],
  "date": "2026-08-08",
  "description": "Dinner at restaurant"
}
```

This is only a reference structure. Students are free to improve the schema.

11. Input Validation

Use Pydantic models to validate API requests.

At minimum:

- Expense title should not be empty.
- Amount should be greater than zero.
- Category should be provided.
- Paid-by member should be provided.
- At least one person should be included in the expense split.
- Required trip information should be provided.

FastAPI should return meaningful validation errors for invalid requests.

12. Error Handling

The application should handle common errors properly.

Examples:

Trip not found

Trip not found

Expense not found

Expense not found

Invalid ID

Invalid trip ID

Invalid amount

Amount must be greater than zero

The application should not crash because of invalid user input.

13. Minimum API Requirements

Students should implement at least the following APIs:

POST /trips

GET /trips/{trip_id}

POST /trips/{trip_id}/expenses

GET /trips/{trip_id}/expenses

GET /expenses/{expense_id}

PUT /expenses/{expense_id}

DELETE /expenses/{expense_id}

GET /trips/{trip_id}/summary

Additional APIs can be created based on the student's design.

14. Technology Requirements

The project must use:

- Python
- FastAPI
- Pydantic
- MongoDB
- PyMongo

Students can use:

- VS Code
- MongoDB Compass
- Swagger UI

15. Testing

Students must test their APIs using FastAPI Swagger UI.

They should test:

- Valid requests
- Invalid requests
- CRUD operations
- Expense filtering
- Sorting
- Expense calculation

- Trip summary

They should include screenshots of successful API execution.

16. Submission Requirements

Submit a GitHub repository containing:

expense-tracker/

|

├── main.py

├── database.py

├── models.py

├── routes/

|

├── requirements.txt

└── README.md

The project structure can be different if the student has a valid reason.

README should contain:

- Project description
- Technologies used
- How to install dependencies
- How to start the FastAPI application
- MongoDB setup

- API list
- Sample request/response
- Screenshots

17. Evaluation Criteria

Area| Focus

FastAPI| API creation and routing

Python| Logic and clean coding

MongoDB| Database design and operations

CRUD| Create, Read, Update, Delete

Querying| Search, filtering and sorting

Expense Split| Correct calculation

Aggregation| Trip/category summaries

Validation| Pydantic validation

Error Handling| Meaningful error responses

Code Quality| Modular and readable code

Important Note

This is a backend assignment. No frontend application is required.

Students are free to decide:

- MongoDB schema

- Project structure
- API response format
- Expense calculation approach
- Settlement algorithm

The focus is not on building a complicated application, but on demonstrating a practical understanding of FastAPI + MongoDB + CRUD + querying + basic business logic.